Concepts

Custom Data

Nautilus Trader supports custom data authored in Python and Rust, and moves that data through the same runtime, persistence, and query pipeline used by the rest of the platform.

This document explains how custom data is:

- Registered at runtime.
- Wrapped across the Python/Rust boundary.
- Serialized to and from Arrow/Parquet.
- Routed through actors and strategies.

Goals

The custom-data architecture satisfies the following requirements:

- Let users define custom data in pure Python without writing Rust code.
- Let Rust-defined custom data use native Rust JSON and Arrow handlers.
- Preserve a single user-facing `CustomData` wrapper at the PyO3 boundary.
- Support persistence in `ParquetDataCatalog` using dynamic type registration instead of hardcoded schemas.
- Make custom data routable through the normal data-engine, actor, and strategy subscription flow.

High-level model

There are two supported authoring modes:

Mode	Example	Registration path	Encode/decode path	Wrap
Pure Python	`@customdataclass_pyo3` class	`register_custom_data_class(...)`	Python callback + Arrow C FFI	`Pyth`
Same-binary Rust	`#[custom_data]` or `#[custom_data(pyo3)]` type	`ensure_custom_data_registered::<T>()` and native extractor	Native Rust	Nativ

Both modes converge on the same outer PyO3 `CustomData` wrapper and the same `DataType` identity model.

End-to-end flow

Core components

`DataRegistry`

`crates/model/src/data/registry.rs` is the central runtime registry module for custom data in the main process. Registration uses atomic `DashMap::entry()` so that concurrent `register_*` and `ensure_*` calls do not race.

The module contains several `OnceLock`-initialized `DashMap` singletons:

- JSON deserializers keyed by `type_name`.
- Arrow schemas, encoders, and decoders keyed by `type_name`.
- Python extractors that convert a Python object into `Arc<dyn CustomDataTrait>`.
- Rust extractor factories that produce Python extractors for same-binary types.

Instead of hardcoding every type into the main binary, Nautilus resolves handlers at runtime using the `type_name` stored in `DataType` and Parquet metadata.

`CustomData`

The outer PyO3 `CustomData` wrapper is the common container that crosses the FFI boundary.

Constructor signature: `CustomData(data_type, data)` where `DataType` comes first, then the inner payload.

It contains:

- A `DataType`.
- An inner custom payload implementing `CustomDataTrait` (wrapped in `Arc<dyn CustomDataTrait>`).

Timestamps (`ts_event`, `ts_init`) are delegated to the inner `CustomDataTrait` implementation and exposed as properties on the wrapper.

On the Python side, `CustomData` exposes value semantics: `__eq__` and `__repr__` are implemented (equality uses the Rust `PartialEq` logic). Instances are intentionally unhashable so that equality remains consistent with the inner payload comparison.

This wrapper is shared across both custom-data modes. User code interacts with one API even though the underlying payload may be:

- A Python-backed wrapper.

- A same-binary Rust value.

`CustomData` JSON envelope

When serialized to JSON (e.g. for `to_json_bytes` / `from_json_bytes`, SQL cache, or Redis), `CustomData` uses a single canonical envelope so that deserialization does not depend on user payload field names:

- `type`: The custom type name (from `CustomDataTrait::type_name`).
- `data_type`: An object with `type_name`, `metadata`, and optional `identifier`.
- `payload`: The inner payload only (the result of `CustomDataTrait::to_json` parsed as a value). Registered deserializers receive only this value in `from_json`, so user structs can use any field names (including `value`) without conflicting with wrapper metadata.

This envelope is produced by Rust `CustomData` serialization and consumed by `DataRegistry` when deserializing custom data from JSON.

`DataType`

`DataType` identifies custom data for routing and persistence.

Constructor: `DataType(type_name, metadata=None, identifier=None)`.

It includes:

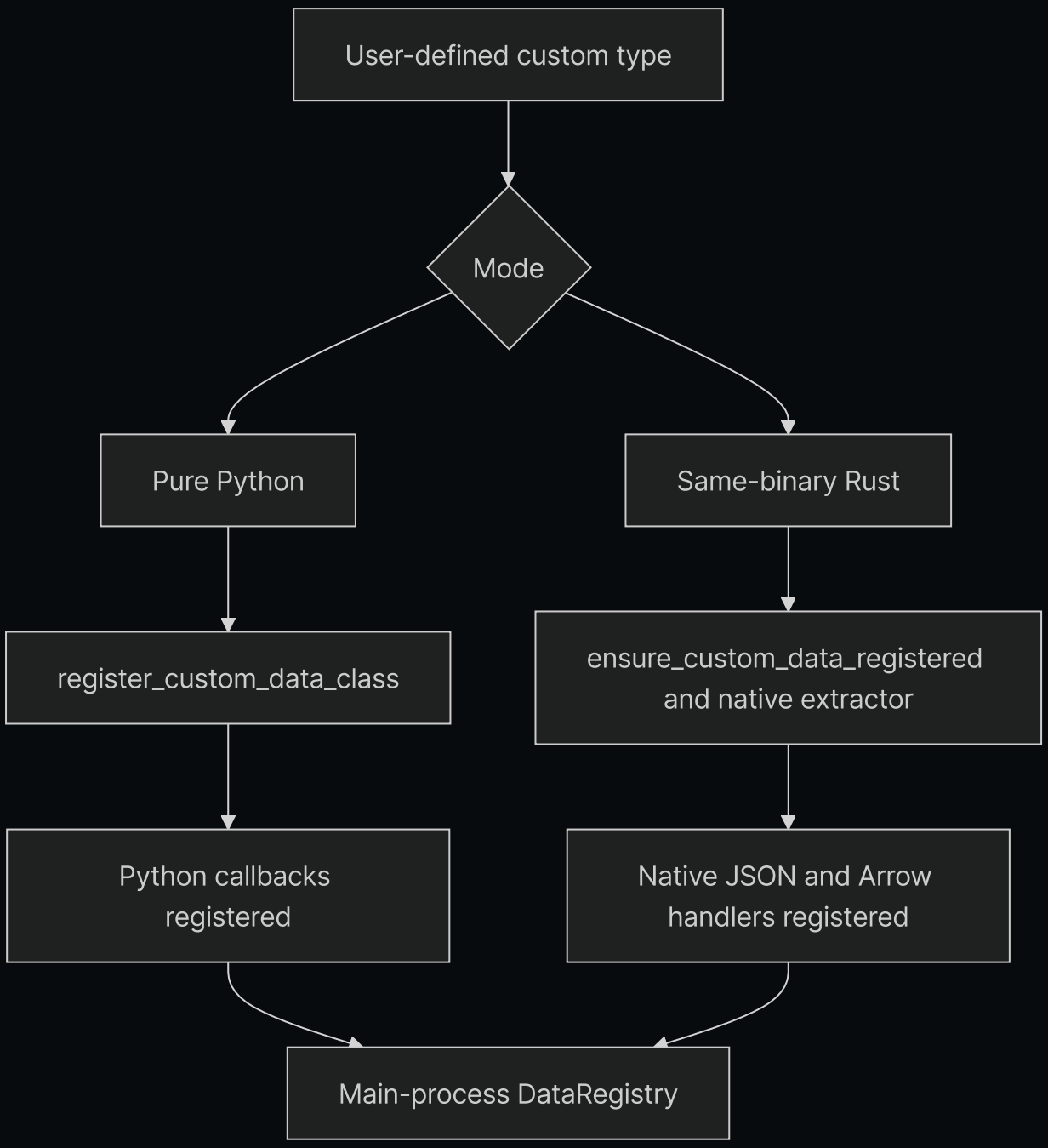
- `type_name`.
- Optional `metadata`.
- Optional `identifier` (used only for catalog pathing, not for routing or equality).

Equality, hashing, and topic routing are derived from `type_name` and `metadata` only. Two `DataType` values with the same type name and metadata but different identifiers compare equal and publish to the same message bus topic. The `identifier` affects only the storage path under `data/custom/<type_name>/<identifier...>`.

Custom-data storage and queries use `DataType`, not just the bare Rust/Python class name. This allows the same logical type to be stored under different metadata or identifiers while still decoding through the same registered handler.

Registration architecture

Registration bridges the gap between Python objects and Rust trait objects.



Pure Python registration

When Python code calls `register_custom_data_class(MyType)`:

1. The type is registered in the Python serialization layer for JSON and Arrow support.
2. Rust registers a Python extractor that wraps Python instances as `PythonCustomDataWrapper`.
3. Rust registers Arrow schema/encode/decode callbacks in `DataRegistry`.

This path is flexible and user-friendly, but Arrow encoding and reconstruction rely on Python callbacks.

Same-binary Rust registration

For Rust types defined inside Nautilus:

1. `#[custom_data]` or `#[custom_data(pyo3)]` generates the necessary trait, JSON, and Arrow implementations.
2. `ensure_custom_data_registered::<T>()` inserts native schema/encoder/decoder handlers into `DataRegistry`.
3. For PyO3-exposed types, a native extractor can convert Python instances back into the concrete Rust type rather than a Python fallback wrapper.

This path stays fully native in Rust for encode/decode.

Registration precedence

`register_custom_data_class(...)` resolves types in the following order:

1. Same-binary native Rust registration.
2. Pure Python fallback registration.

That ordering preserves the fastest available path for types already known natively by the main binary.

Wrapper backends

Internally, the outer `CustomData` wrapper can hold different payload implementations.

`PythonCustomDataWrapper`

Used for pure Python custom data.

Responsibilities:

- Stores a reference to the Python object.

- Caches `ts_event`, `ts_init`, and `type_name`.
- Implements `CustomDataTrait`.
- Calls Python methods for JSON and Arrow-related operations under the GIL.

This is the fallback path when the main process does not have a native Rust representation for the type.

Native same-binary Rust payload

For Rust types compiled into Nautilus, the inner payload is the concrete Rust type itself and can be downcast directly from `Arc<dyn CustomDataTrait>`.

No Python callback path is needed for serialization or decode.

Persistence architecture

Why dynamic Arrow registration is needed

Built-in Nautilus data types have schemas and encoders known statically to the Rust binary. Custom data does not. The persistence layer therefore resolves custom data dynamically using the registered `type_name`.

Catalog write flow

`ParquetDataCatalog` expects custom writes to come in as `CustomData` values.

The custom-data write path:

1. Extracts `type_name`, `metadata`, and `identifier` from `DataType`.
2. Looks up the Arrow encoder in `DataRegistry`.
3. Encodes the values to a `RecordBatch`.
4. Appends a `data_type` column containing the persisted `DataType`.
5. Attaches `type_name` and metadata to the Arrow schema.
6. Writes the batch to Parquet under the custom-data path.

The path layout is:

- `data/custom/<type_name>/<identifier...>`

Identifiers are normalized before becoming path segments.

Catalog read flow

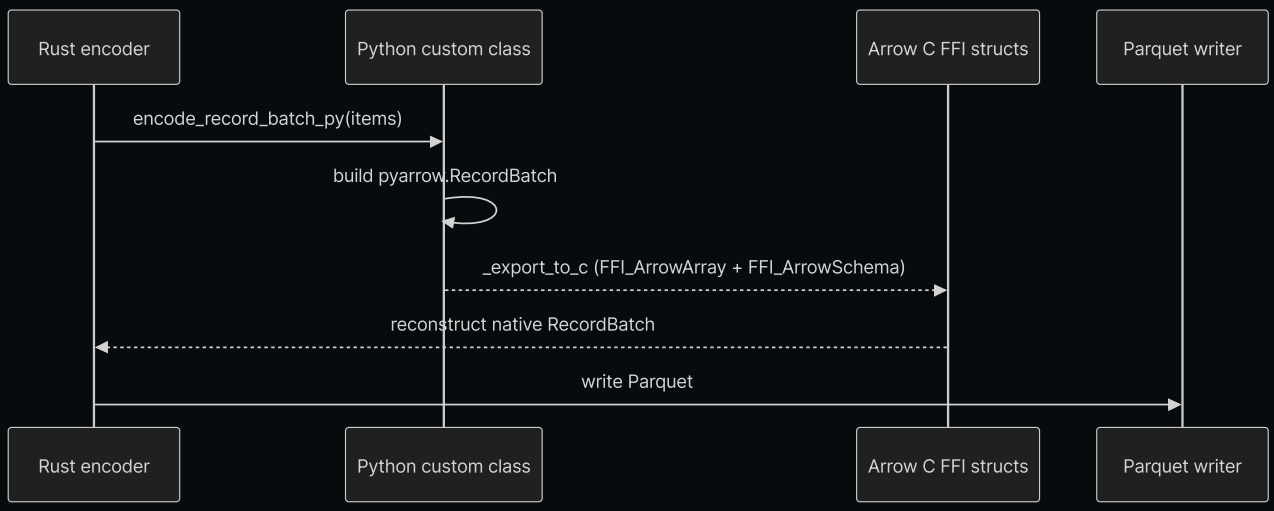
On query:

1. The catalog reads matching Parquet files.
2. Extracts `type_name` from schema metadata.
3. Asks `DataRegistry` for the registered decoder.
4. Decodes the `RecordBatch` into `Vec<Data>`.
5. Reconstructs `CustomData` with the original `DataType`.

This makes custom-data query resolution symmetric with write-time registration. When converting a Feather stream to Parquet (e.g. after a backtest), the custom-data branch decodes batches and writes them via `write_custom_data_batch` so that custom data written through the Feather writer is correctly converted to Parquet.

The Arrow C FFI bridge

Pure Python custom data cannot provide native Rust Arrow encode logic directly. For those types, Nautilus uses the Arrow C FFI interface to pass `RecordBatch` data between Python and Rust without serialization overhead.



Pure Python encode path

For pure Python classes:

1. Rust acquires the GIL.
2. Rust calls `encode_record_batch_py(...)` on the Python class.
3. Python converts objects to a `pyarrow.RecordBatch`.
4. Python exports the batch via `_export_to_c` into Arrow C FFI structs.
5. Rust reconstructs a native `RecordBatch` from the FFI structs and writes it.

Pure Python decode path

For the reverse direction:

1. Rust converts its `RecordBatch` into Arrow C FFI structs.
2. Python imports the batch via `RecordBatch._import_from_c`.
3. Python calls `decode_record_batch_py(metadata, batch)` on the class.
4. Rust wraps the returned Python objects in `PythonCustomDataWrapper`.

Native paths

The Arrow C FFI bridge is not used for same-binary Rust custom data. Those types use native Rust encode/decode handlers registered in the main process.

Reconstruction on query

When custom data is loaded back from the catalog, reconstruction depends on the backend:

- Same-binary Rust types decode directly to native Rust values.
- Pure Python types reconstruct through the registered Python class using `from_dict` or `from_json`.

In all cases the caller receives the same outer `CustomData` wrapper at the PyO3 API boundary.

Runtime integration

Custom data is not only a persistence feature. It also participates in Nautilus runtime routing.

Relevant integrations include:

- `crates/data/src/engine/mod.rs` publishes `CustomData` through the message bus.
- `crates/common/src/msgbus/switchboard.rs` derives custom topics from `DataType`.
- `crates/common/src/actor/*` routes custom data into actor subscriptions.
- `crates/trading/src/python/strategy.rs` exposes custom data to Python strategy `on_data`.
- `crates/backtest/src/engine.rs` treats `Data::Custom` as data-engine-delivered input rather than exchange-routed data.

A registered custom type can be persisted, queried, subscribed to, and consumed through the same runtime interfaces as other data families.

SQL cache and database integration

The SQL cache/database layer also supports `CustomData`.

Current behavior:

- PostgreSQL stores custom data in the `custom` table.
- The stored record includes `data_type`, `metadata`, `identifier`, and full JSON payload.
- Reads reconstruct `CustomData` using `CustomData::from_json_bytes(...)`.
- Python SQL bindings expose `add_custom_data` and `load_custom_data`.
- Redis cache stores custom data under keys `custom:<ts_init_020>:<uuid>` with full `CustomData` JSON as value.
- Redis `add_custom_data` and `load_custom_data` filter by `DataType` (type_name, metadata, identifier) and return results sorted by `ts_init`; this is exposed via the PyO3 `RedisCacheDatabase` API.

Cython custom data

The Cython `@customdataclass` system is separate from this architecture. This document describes the PyO3 custom-data system:

- PyO3 `CustomData`.
- Dynamic runtime registration.
- Arrow/Parquet persistence.
- Native Rust execution paths.

Practical implications

This architecture gives Nautilus two important properties:

1. Python-first extensibility for users who only want to write Python.
2. Native Rust performance for built-in or compiled custom types.

The result is one conceptual custom-data system with two backends, rather than separate feature silos for Python-only and Rust-only data types.

‹ Greeks
Previous Page

Order Book ›
Next Page